

Smart BeeHive

Analisi funzionale del sistema:

Piattaforma Web per il Monitoraggio Remoto di Arnie Intelligenti

Zara Luigi - Classe 5E

A.S. 2025/26

Sommario

1. Introduzione.....	3
2. Obiettivi del Sistema.....	3
3. Attori del Sistema.....	3
3.2 Piattaforma IoT (ThingsBoard).....	4
3.3 Backend PHP (Proxy API).....	4
3.4 Sensori e Gateway Hardware.....	4
4. Funzionalità Principali.....	4
4.1 Dashboard Multi-Arnia (index.php).....	4
4.1.1 Statistiche aggregate.....	4
4.1.2 Griglia delle arnie.....	5
4.1.3 Pannello laterale con tre tab.....	5
4.2 Dettaglio Singola Arnia (arnie.php).....	5
4.2.1 Indicatori attuali.....	5
4.2.2 Indicatore di stato sensori (semaforo).....	5
4.2.3 Correlazione R^2 (Pearson).....	5
4.2.4 Storico allarmi dell'arnia.....	6
4.2.5 Grafici storici e selettore temporale.....	6
4.3 Gestione Allarmi (allarmi.php).....	6
4.3.1 Caricamento e mapping degli allarmi.....	6
4.3.2 Riepilogo statistico.....	6
4.3.3 Filtri per stato.....	6
4.3.4 Gestione dello stato tramite modale.....	6
4.3.5 Comportamento automatico degli stati.....	7
4.4 Archivio Allarmi Risolti (archivio.php).....	7
4.5 Modalità Demo (Mock Data).....	7
5. Flussi Operativi.....	7
5.1 Flusso dei Dati.....	7
5.2 Interazione Utente-Sistema.....	8
6. Modello dei Dati.....	8
7. Logica degli Allarmi.....	9
7.1 Generazione degli Allarmi.....	9
7.2 Stati degli Allarmi.....	9
7.3 Impatto sull'Interfaccia.....	9
8. Gestione degli Utenti e Sicurezza.....	10
8.1 Sistema di Autenticazione.....	10
8.2 Ruoli e Permessi.....	10
8.3 Funzionalità di Impersonazione.....	10
8.4 Reset Password da Admin.....	11
8.5 Gestione Profilo Utente (profile.php).....	11
8.6 Modalità Offline del Database.....	11
9. Interfaccia Utente.....	11
9.1 Navbar e Sidebar.....	11

9.2 Dashboard (index.php).....	12
9.3 Dettaglio Arnia (arnie.php).....	12
9.4 Gestione Allarmi (allarmi.php).....	12
9.5 Archivio Allarmi (archivio.php).....	12
10. Funzionalità di Analisi dei Dati.....	12
10.1 Panoramica Multi-Arnia.....	12
10.2 Analisi Avanzata.....	12
11. Requisiti Funzionali.....	13
12. Requisiti Non Funzionali.....	14
12.1 Prestazioni.....	14
12.2 Sicurezza.....	14
12.3 Scalabilità.....	14
13. Conclusione.....	15

1. Introduzione

Smart BeeHive è una piattaforma software di monitoraggio remoto per apiari composti da più arnie sensorizzate. Il sistema nasce per poter supervisionare lo stato di salute delle colonie, il livello di produzione del miele e le condizioni ambientali interne ed esterne all'arnia, senza necessità di ispezione fisica continua.

Il contesto applicativo è quello dell'apicoltura di precisione (smart beekeeping), in cui sensori IoT raccolgono dati da ciascuna arnia e li trasmettono a una piattaforma centralizzata. I dati vengono poi esposti tramite un'interfaccia web personalizzata, accessibile da desktop, tablet e smartphone.

Il monitoraggio non è continuo in senso stretto: i sensori operano a blocchi e inviano i dati a intervalli regolari (circa ogni 8 ore) o in modo immediato in presenza di valori critici. Il sistema è pertanto classificabile come near real-time. Nonostante ciò, il frontend aggiorna periodicamente la vista ogni 30 secondi per garantire un feedback tempestivo all'utente.

2. Obiettivi del Sistema

Il sistema persegue tre macro-obiettivi funzionali:

- **Monitoraggio ambientale e produttivo:** raccogliere e visualizzare in modo chiaro le misurazioni di temperatura interna ed esterna, umidità, peso del miele e frequenza acustica di picco per ogni arnia dell'apiario.
- **Prevenzione e gestione delle anomalie:** rilevare automaticamente condizioni fuori soglia tramite il sistema di allarmi di ThingsBoard, notificarle all'utente nell'interfaccia web e consentirne la gestione (presa in carico, risoluzione, annotazione).
- **Supporto decisionale:** fornire strumenti di analisi storica (grafici temporali su finestre di 24 ore, 7 giorni, 30 giorni e 1 anno, derivate della temperatura e del peso, correlazione statistica R^2 tra temperatura interna ed esterna) utili a identificare tendenze e prendere decisioni informate sulla gestione dell'apiario.

3. Attori del Sistema

Il sistema coinvolge tre categorie di attori principali, ognuna con ruoli e responsabilità distinte.

3.1 Utente (Apicoltore / Operatore)

L'utente finale è tipicamente un apicoltore o un tecnico che accede alla dashboard tramite browser. Il codice prevede tre livelli di ruolo (admin, operator e viewer) gestiti tramite un database MySQL con hashing delle password (password_hash / password_verify). Il ruolo dell'utente determina i permessi: il viewer può solo consultare i dati; l'operator può gestire gli allarmi; l'admin può creare nuovi account.

3.2 Piattaforma IoT (ThingsBoard)

ThingsBoard è il backend IoT del sistema. Si occupa della ricezione e archiviazione dei dati telemetrici provenienti dai sensori hardware, della generazione automatica degli allarmi tramite regole configurabili e dell'esposizione di API REST che il layer PHP interroga tramite chiamate cURL. Il sistema non utilizza le dashboard native di ThingsBoard: l'interfaccia utente è completamente sviluppata dal team di progetto.

ThingsBoard gestisce anche l'autenticazione del backend: il file `api.php` ottiene un token JWT tramite chiamata POST a `/api/auth/login`, lo salva in sessione PHP con una cache di 3600 secondi e lo riutilizza per tutte le richieste successive.

3.3 Backend PHP (Proxy API)

Il layer intermedio, composto principalmente da `api.php` e `alarm_action.php`, funge da proxy tra il frontend JavaScript e ThingsBoard. Le sue responsabilità includono: nascondere le credenziali di ThingsBoard (non esposte al client), implementare una cache di sessione di 5 secondi per le risposte aggregate, eseguire chiamate multi-curl parallele per recuperare le telemetrie di tutte le arnie in un'unica richiesta e calcolare i flag `is_stale` e `is_very_stale` per segnalare dati rispettivamente più vecchi di 12 e 24 ore.

3.4 Sensori e Gateway Hardware

Ogni arnia è dotata di sensori per temperatura, umidità, peso e attività sonora. Una centralina locale basata su ESP32 acquisisce le misurazioni, elabora localmente il segnale audio estraendo parametri come l'intensità e la frequenza di picco, e trasmette i dati a ThingsBoard tramite comunicazione LoRa verso un gateway connesso a Internet. Tra un invio e l'altro il dispositivo può entrare in modalità deep sleep per ridurre i consumi. Il frontend riceve direttamente valori scalari (ad esempio `peakFreq`), senza necessità di elaborare il segnale grezzo.

4. Funzionalità Principali

4.1 Dashboard Multi-Arnia (`index.php`)

La dashboard principale offre una visione d'insieme dell'intero apiario su un'unica schermata. Al caricamento, il frontend recupera i dati di tutte le arnie con una singola chiamata a `api.php` (chiamata a `tbLoadAllHives`), dopodiché esegue un polling automatico ogni 30 secondi per mantenere la vista aggiornata.

4.1.1 Statistiche aggregate

Nella parte superiore della pagina sono presenti cinque indicatori aggregati, calcolati lato client dalla funzione `computeStats()`: peso totale del miele (somma dei pesi di tutte le arnie in kg), numero di allarmi attivi, temperatura interna media delle arnie attive, umidità media e temperatura esterna (primo valore non nullo disponibile tra le arnie). Questi valori vengono aggiornati a ogni ciclo di polling.

4.1.2 Griglia delle arnie

Ogni arnia è rappresentata da una card cliccabile che mostra: nome identificativo, stato di salute (ONLINE, ATTENZIONE o OFFLINE) con colore distintivo, indicatore grafico verticale del livello di miele, e quattro metriche numeriche (peso, frequenza di picco, temperatura, umidità) con colorazione condizionale in base alle soglie. Facendo clic su una card, l'utente viene reindirizzato alla pagina di dettaglio dell'arnia corrispondente.

La logica di colorazione delle metriche è implementata nelle funzioni `getTempColor`, `getHumColor`, `getWeightColor` e `getFreqColor`. Il sistema distingue due soglie di staleness: i dati tra 12 e 24 ore di inattività vengono segnalati con stato giallo (flag `is_stale = true`), mentre i dati superiori a 24 ore vengono segnalati con stato rosso (flag `is_very_stale = true`).

4.1.3 Pannello laterale con tre tab

A destra della griglia delle arnie si trova un pannello a schede che include: Storico Allarmi (attivo di default), Panoramica (grafici storici multi-arnia) e Analisi Avanzata (correlazione R^2 e derivate). Il selettore temporale 24h / 7 giorni / 1 mese / 1 anno appare dinamicamente quando si passa alla tab Panoramica o Analisi Avanzata.

4.2 Dettaglio Singola Arnia (arnie.php)

Accessibile tramite URL con parametro `id`, questa pagina offre un'ispezione approfondita di una specifica arnia. I dati vengono recuperati all'avvio con `tbGetTelemetry` per l'intervallo "24h"; se non sono disponibili dati nelle ultime 24 ore, il sistema effettua una seconda chiamata con modalità "latest" per recuperare l'ultimo valore assoluto, impostando automaticamente `is_stale = true`.

4.2.1 Indicatori attuali

Quattro riquadri mostrano i valori istantanei più recenti: temperatura interna, umidità, peso e frequenza di picco. Tutti e quattro applicano la colorazione condizionale tramite le funzioni `applyMetricColors`. Un ulteriore elemento verticale a serbatoio (honey tank) mostra graficamente il livello percentuale del miele (`honeyPct`), con animazione CSS fluida.

4.2.2 Indicatore di stato sensori (semaforo)

Un riquadro testuale mostra lo stato di salute dell'arnia in tre varianti: verde (Ottimale) se i dati sono recenti e nella norma; giallo (Instabile) se i dati superano le 12 ore di inattività (`is_stale`); rosso (Offline) se i dati superano le 24 ore (`is_very_stale`), se i valori sono a zero, la temperatura è fuori dalle soglie critiche ($< -5^{\circ}\text{C}$ o $> 40^{\circ}\text{C}$). Il messaggio riporta anche il timestamp dell'ultimo aggiornamento nel formato "dd/mm HH:mm".

4.2.3 Correlazione R^2 (Pearson)

Nella sezione Stato Sensori è visualizzato il coefficiente di determinazione R^2 calcolato dalla funzione `computeR2` tra temperatura interna e temperatura esterna. I dati vengono abbinati

per prossimità temporale (tolleranza massima di 1 ora tra timestamp). Se i punti abbinati sono meno di 3, viene mostrata la dicitura "Dati insuff.".

4.2.4 Storico allarmi dell'arnia

Un pannello laterale elenca gli allarmi filtrati per la specifica arnia, caricati tramite `tbLoadAlarms` e filtrati per nome dispositivo. Ogni allarme mostra messaggio, timestamp e stato (DA GESTIRE / APERTO / RISOLTO) con il badge colorato corrispondente. Il pannello ha altezza fissa e scrollbar personalizzata.

4.2.5 Grafici storici e selettore temporale

Quattro grafici `Chart.js` visualizzano lo storico della singola arnia: temperatura interna vs. esterna (linea doppia con legenda), frequenza di picco in Hz (linea), variazione del peso nel tempo (barre), umidità interna (linea). Un selettore a schede (24h, 7 giorni, 1 mese, 1 anno) permette di ricaricare i grafici con intervalli diversi senza ricaricare la pagina. Durante il caricamento, i canvas vengono temporaneamente messi a opacità ridotta a indicare il refresh.

4.3 Gestione Allarmi (`allarmi.php`)

La pagina di gestione degli allarmi presenta tutti gli allarmi provenienti da ThingsBoard, organizzati per arnia di appartenenza in sezioni visivamente distinte.

4.3.1 Caricamento e mapping degli allarmi

Al caricamento, la funzione `refreshAlarms` recupera i dati da `api.php` e mappa i tipi di allarme ThingsBoard (stringhe tecniche come `HoneyReady`, `DeviceDifferentTemperature`, ecc.) in etichette leggibili in italiano tramite il dizionario `alarmTypeLabels`.

4.3.2 Riepilogo statistico

Nella parte superiore della pagina, tre contatori mostrano quanti allarmi sono in stato "Da Gestire", "Aperti" e "Risolti". I contatori si aggiornano in tempo reale al variare dei filtri e delle azioni utente.

4.3.3 Filtri per stato

Una barra di filtri permette di visualizzare: tutti gli allarmi, solo quelli "Da Gestire", solo quelli "Aperti" o solo quelli "Risolti". Ogni filtro è applicato lato client e la funzione `applyFilterRender` raggruppa gli allarmi per arnia e renderizza solo le sezioni non vuote.

4.3.4 Gestione dello stato tramite modale

Facendo clic sul pulsante di stato di un allarme, si apre un pop-up modale che permette di: visualizzare titolo e dati dell'allarme, selezionare il nuovo stato ("Da Gestire" o "Risolto"), aggiungere una nota testuale (facoltativa) e confermare il salvataggio.

Al salvataggio, la funzione `saveAlarmStatus` esegue una chiamata POST a `alarm_action.php`, che a sua volta chiama il corrispondente endpoint ThingsBoard (`/api/alarm/{id}/ack` per "Aperto", `/api/alarm/{id}/clear` per "Risolto"). Lo stato viene contestualmente salvato in `localStorage` con la chiave `alarmStates`, garantendo persistenza locale tra le sessioni browser.

Una notifica appare in basso a destra per confermare l'avvenuto salvataggio.

4.3.5 Comportamento automatico degli stati

Il sistema implementa una logica di transizione automatica: quando un utente apre la modale di un allarme in stato "Da Gestire", questo viene immediatamente promosso a "Aperto" anche se l'utente chiude il pop-up senza salvare esplicitamente. Questo meccanismo simula la "presa visione" dell'allarme. In modalità demo, le chiamate a ThingsBoard non vengono effettuate e il salvataggio è solo locale.

4.4 Archivio Allarmi Risolti (archivio.php)

La pagina archivio è una vista dedicata esclusivamente agli allarmi che si trovano nello stato "Risolto", separata e distinta dalla pagina di gestione allarmi (`allarmi.php`). È accessibile dalla sidebar di navigazione.

Al caricamento, la funzione `refreshArchive` recupera tutti gli allarmi tramite `api.php` e li filtra lato client mantenendo solo quelli il cui stato effettivo (determinato da `getEffectiveAlarmStatus`) è "closed". Gli allarmi vengono raggruppati per arnia e visualizzati in sezioni analoghe a quelle di `allarmi.php`, con un contatore del numero di allarmi risolti nella parte superiore.

La pagina include la modalità demo, che carica un array `mockArchiveData` con allarmi di esempio già in stato risolto, e un pulsante di aggiornamento manuale. Il modulo JS dedicato è `archivio.js`, che dipende da `alarm_state.js` per la gestione dello stato condiviso.

4.5 Modalità Demo (Mock Data)

Ogni pagina dell'applicazione include un interruttore toggle "Modalità Demo" in alto a destra. Quando attivato, lo stato viene persistito in `localStorage` (chiave `mockMode`). In questa modalità, il frontend sostituisce completamente le chiamate API con dati statici predefiniti.

In modalità demo: la dashboard carica dati fittizi da `hivesData` con valori predefiniti per ciascuna arnia; i grafici vengono popolati con serie temporali generate sinteticamente con piccole variazioni casuali; la pagina allarmi carica l'array `mockAlarmsData` con 8 allarmi di esempio distribuiti su 5 arnie; la pagina archivio carica `mockArchiveData` con allarmi già risolti; la pagina dettaglio arnia genera una telemetria mock con 5 punti per ogni grandezza nell'arco delle ultime 4 ore. In modalità demo la navbar non esegue il polling degli status dot.

5. Flussi Operativi

5.1 Flusso dei Dati

Il percorso dei dati dal campo all'interfaccia utente si articola nei seguenti passaggi:

1. I sensori hardware misurano temperatura, umidità, peso e attività sonora a intervalli regolari. La centralina basata su ESP32 elabora localmente i segnali (in particolare i parametri acustici) e trasmette i dati tramite rete LoRa a un gateway Raspberry Pi posizionato in prossimità delle arnie, dotato di modulo LoRa HAT. Il gateway raccoglie i pacchetti di tutte le arnie e li trasmette via radio a un secondo dispositivo, un Raspberry Pi ricevitore con antenna esterna collocato sul lato dell'edificio più vicino all'apiario, che provvede all'inoltro verso ThingsBoard tramite richieste HTTP.
2. ThingsBoard archivia i dati nella propria base dati time-series ed esegue rule chains per la validazione delle telemetrie, la generazione di allarmi e il calcolo di indicatori statistici (come medie e variazioni nel tempo).
3. Il backend PHP (api.php) autentica la sessione con ThingsBoard (token JWT in cache), esegue chiamate REST parallele tramite cURL, normalizza i dati ricevuti, calcola indicatori derivati come il flag is_stale e restituisce un unico JSON contenente telemetrie e allarmi.
4. Il frontend JavaScript elabora la risposta, aggiorna dinamicamente l'interfaccia utente, applica logiche di visualizzazione condizionale e segnala eventuali anomalie o dati non aggiornati.

5.2 Interazione Utente-Sistema

Il flusso tipico di un utente che gestisce un allarme è il seguente: l'utente accede alla dashboard, nota il badge rosso con il conteggio degli allarmi attivi, clicca su "Gestisci Allarmi", visualizza la lista organizzata per arnia, clicca sul pulsante di stato dell'allarme, legge i dettagli nel pop-up, eventualmente aggiunge una nota, seleziona lo stato "Risolto" e conferma. Il sistema invia la richiesta a ThingsBoard e aggiorna la UI con lo stato di conferma.

6. Modello dei Dati

Il sistema monitora i seguenti parametri telemetrici per ciascuna arnia, identificati dalle chiavi usate nelle chiamate API:

Parametro	Significato Funzionale
tempIn	Temperatura interna dell'arnia (°C). L'intervallo ottimale per le api è 33–38°C. Valori fuori da questo range indicano stress termico della colonia.
tempOut	Temperatura esterna ambientale (°C). Utilizzata per il calcolo della correlazione R^2 con la temperatura interna e per i grafici comparativi.
humidity	Umidità relativa interna (%). Range ottimale 40–65%. Valori troppo alti o troppo bassi possono compromettere la qualità del miele e la salute delle api.
honeyWeightKg	Peso totale dell'arnia in chilogrammi. Indica indirettamente la quantità di

	miele prodotto e il peso della colonia. Un calo rapido può segnalare sciamatura o furto di miele.
honeyPct	Percentuale di riempimento del serbatoio del miele (0–100%). Calcolata da ThingsBoard, viene visualizzata tramite l'indicatore grafico a serbatoio verticale
peakFreq	Frequenza di picco dell'attività sonora (Hz). Calcolata da ThingsBoard tramite FFT sul segnale acustico. Range normale 220–270 Hz. Valori superiori a 270 Hz indicano stress; sopra 380 Hz possono segnalare sciamatura imminente.

7. Logica degli Allarmi

7.1 Generazione degli Allarmi

Gli allarmi sono generati automaticamente da ThingsBoard in base a regole configurate sul backend. Il frontend non genera allarmi autonomamente: si limita a recuperarli e a visualizzarli. Il codice gestisce varie tipologie di allarme distinte, tra cui: HoneyReady (miele pronto da raccogliere), DeviceOldTemperature/Humidity/Weight/NoiseFrequency (dati non aggiornati), DeviceDifferentTemperature/Humidity/Weight/NoiseFrequency/NoiseIntensity (anomalie comparative tra arnie), ErrorDeviceTimeseries, TelemetryInvalidKey, FailedAssetAttributes ed ErrorTimeSeriesWeightDevice.

7.2 Stati degli Allarmi

Ogni allarme può trovarsi in uno dei seguenti stati logici, gestiti con un sistema ibrido che combina lo stato ThingsBoard con un override locale:

- Da Gestire (system): stato iniziale di un allarme appena rilevato da ThingsBoard (ACTIVE_UNACK). Rappresenta un evento non ancora preso in carico da nessun operatore.
- Aperto (open): l'operatore ha visualizzato l'allarme (corrisponde a ACTIVE_ACK su ThingsBoard). La transizione avviene automaticamente all'apertura della modale di gestione.
- Risolto (closed): la situazione è normalizzata (corrisponde a CLEARED_ACK o CLEARED_UNACK su ThingsBoard). L'operatore può aggiungere una nota esplicativa.

La logica è centralizzata nel modulo alarm_state.js (funzione tbStatusToUI), condiviso tra tutte le pagine che visualizzano allarmi (index.js, allarmi.js, arnie.js, archivio.js).

7.3 Impatto sull'Interfaccia

Gli allarmi attivi impattano sulla UI in più punti: il contatore "Allarmi" nella dashboard principale viene aggiornato dalla funzione computeStats contando gli allarmi in stato system o open; le card delle arnie mostrano un punto colorato (verde, giallo, rosso) in base allo stato determinato dal flag is_stale e dai valori delle metriche; la pagina allarmi raggruppa visivamente gli eventi per arnia con badge che indicano quanti allarmi richiedono attenzione.

8. Gestione degli Utenti e Sicurezza

8.1 Sistema di Autenticazione

L'accesso all'applicazione è protetto da un doppio livello di autenticazione. Il primo livello è un'autenticazione HTTP Basic configurata tramite file `.htaccess`, che richiede credenziali prima ancora che venga caricata qualsiasi pagina PHP. Il secondo livello è il modulo di login PHP (`login.php`), che verifica le credenziali dell'utente sul database MySQL tramite `password_hash` e `password_verify` (algoritmo BCrypt).

Una volta autenticato, la sessione PHP mantiene l'ID utente, il nome e il ruolo. Ogni pagina protetta include `auth.php`, che verifica la sessione e reindirizza al login in caso di accesso non autorizzato.

8.2 Ruoli e Permessi

Il sistema implementa tre ruoli con gerarchia crescente, verificata sia lato client che lato server tramite la funzione `requireRole()`:

Ruolo	Permessi
viewer	Può consultare dashboard, dettaglio allarmi e lista allarmi. Non può modificare lo stato degli allarmi (bloccato anche server-side in <code>alarm_action.php</code> con risposta HTTP 403).
operator	Tutti i permessi del viewer, più la gestione degli allarmi (apertura, risoluzione, annotazione).
admin	Tutti i permessi dell'operator, più accesso alla gestione utenti (creazione, modifica ruolo, eliminazione, reset password) e alla funzionalità di impersonazione.

8.3 Funzionalità di Impersonazione

Gli amministratori possono accedere all'interfaccia come se fossero un altro utente registrato, tramite il file `impersonate.php`. Questa funzionalità è disponibile dalla pagina di gestione utenti (`utenti.php`) ed è riservata al ruolo admin.

Al momento dell'impersonazione, la sessione corrente dell'admin viene salvata in `$_SESSION['original_admin']`, quindi i dati di sessione vengono sovrascritti con quelli dell'utente target. Durante la sessione di impersonazione, la navbar rileva la presenza di `$_SESSION['original_admin']` e mostra un banner di avviso ambra in cima a tutte le pagine, indicando il nome dell'utente impersonato e offrendo un pulsante per terminare l'impersonazione (`stop_impersonate.php`), che ripristina la sessione originale dell'admin.

La funzionalità non è disponibile in modalità offline (quando il database MySQL non è raggiungibile) e non consente a un admin di impersonare sé stesso.

8.4 Reset Password da Admin

Il file `reset_password_admin.php` consente agli admin di reimpostare la password di qualsiasi altro utente. La richiesta viene validata server-side: viene verificato che l'utente target esista, che la nuova password sia di almeno 8 caratteri, che i due campi di conferma coincidano e che l'admin non stia cercando di reimpostare la propria password (per la quale esiste la pagina Profilo).

8.5 Gestione Profilo Utente (`profile.php`)

Ogni utente può accedere alla propria pagina profilo, che mostra nome, ruolo e data di iscrizione. Da questa pagina è possibile modificare la propria password inserendo quella attuale e confermando quella nuova (minimo 8 caratteri). La verifica avviene tramite `password_verify` prima dell'aggiornamento. In modalità offline la pagina mostra un messaggio di avviso disabilitando il modulo di cambio password.

8.6 Modalità Offline del Database

Il file `db.php` implementa una logica di resilienza per la connessione MySQL: se il tentativo di connessione fallisce, lo stato "offline" viene registrato in sessione con il timestamp del primo fallimento. Per i successivi 10 minuti (600 secondi), il sistema non ritenta la connessione, definendo la costante `OFFLINE_MODE` come true. Trascorso l'intervallo, viene eseguito un nuovo tentativo.

Le pagine che dipendono dal database (`utenti.php`, `profile.php`) mostrano messaggi di avviso specifici in modalità offline e disabilitano le operazioni che richiedono il DB. Alla riconnessione la modalità online viene ripristinata automaticamente.

9. Interfaccia Utente

L'interfaccia adotta un design Dark Theme con effetti glassmorphism (sfondi semitrasparenti) e una palette cromatica coerente: ambra/oro per il miele, verde per lo stato ottimale, arancio per i warning, rosso per gli allarmi e blu per i dati esterni. Le icone sono fornite dalla libreria Lucide. Il layout è responsive tramite Bootstrap 5.3.2 e si adatta a schermi mobili con breakpoint specifici in CSS.

9.1 Navbar e Sidebar

La navigazione è gestita da una sidebar laterale retraibile, presente in tutte le pagine tramite `includes/navbar.php`. La sidebar contiene i link alle pagine principali (Dashboard, Allarmi, Archivio, Utenti, Profilo), una voce per ogni arnia monitorata con relativo status dot colorato (verde/giallo/rosso/offline), e le informazioni sull'utente corrente.

I status dot nella sidebar vengono aggiornati ogni 30 secondi tramite un polling indipendente gestito da `navbar.js` (funzione `updateHivesDots`), che chiama `api.php` e ricalcola lo stato di ogni arnia. In modalità demo il polling è disabilitato. Su mobile, la sidebar si apre con il pulsante "hamburger" (☰) e si chiude tramite overlay o tasto ESC.

9.2 Dashboard (index.php)

La schermata è strutturata con un header fisso contenente il titolo, il toggle della modalità demo e il nome utente, seguito da una riga di cinque card statistiche aggregate. Il corpo è diviso in due colonne: a sinistra la griglia delle arnie (30% della larghezza), a destra il pannello con le tre tab (70%). Su dispositivi mobili le due sezioni si impilano verticalmente con la colonna destra sopra.

9.3 Dettaglio Arnia (arnie.php)

La pagina è strutturata in una sezione superiore con la colonnina del miele, quattro riquadri metrici, il pannello di stato sensori e lo storico allarmi. Sotto si trovano il selettore temporale e i quattro grafici Chart.js disposti in griglia 2x2. L'altezza del pannello storico allarmi è fissa per allinearsi visivamente alla colonna sinistra su desktop.

9.4 Gestione Allarmi (allarmi.php)

La pagina è organizzata in: riquadro statistiche (Da Gestire, Aperti, Risolti), barra filtri orizzontale con toggle demo e bottone aggiorna, e area principale con le sezioni per arnia. Ogni sezione mostra un header con nome arnia e badge conteggio, seguito dalle righe degli allarmi.

9.5 Archivio Allarmi (archivio.php)

La pagina mostra un header con il contatore degli allarmi risolti, una barra strumenti con toggle demo e pulsante di aggiornamento, e l'area principale con le sezioni per arnia. Lo stile grafico è coerente con allarmi.php ma i badge riportano il conteggio degli allarmi risolti per ogni arnia.

10. Funzionalità di Analisi dei Dati

10.1 Panoramica Multi-Arnia

Nella tab Panoramica della dashboard sono presenti quattro grafici che aggregano i dati di tutte le arnie: andamento temperature (una linea per arnia più la media generale), livelli di umidità media, peso totale dell'apiario e frequenza sonora media. I dati vengono arrotondati a blocchi di 15 minuti (funzione roundTs) per facilitare l'allineamento temporale tra arnie diverse. Le serie aggregate usano le funzioni aggregateMean e aggregateSum.

10.2 Analisi Avanzata

La tab Analisi Avanzata include tre grafici specializzati. Il grafico a dispersione (scatter plot) visualizza la correlazione tra temperatura interna ed esterna, abbinando i punti con una tolleranza temporale di 30 minuti. Il valore R^2 viene calcolato e visualizzato numericamente sopra il grafico. Il grafico a barre "Velocità Variazione Temperatura" mostra la derivata prima della temperatura media ($^{\circ}\text{C}/\text{h}$) con barre ambra per il riscaldamento e blu per il raffreddamento. Il grafico "Rateo Variazione Peso" mostra la somma delle derivate dei pesi di tutte le arnie (kg/h) con barre verdi per produzione e rosse per perdita.

La funzione `computeDerivative` calcola la derivata discreta di una serie temporale come rapporto incrementale ($\frac{\Delta \text{valore}}{\Delta t}$), espresso in unità orarie. Questa funzione è condivisa tra la tab Analisi Avanzata della dashboard e la pagina di dettaglio `arnia.php`.

11. Requisiti Funzionali

Di seguito sono elencati i requisiti funzionali:

- **RF01** – Il sistema deve recuperare e visualizzare le ultime telemetrie disponibili per ogni arnia configurata.
- **RF02** – Il sistema deve aggiornare automaticamente la dashboard ogni 30 secondi tramite polling.
- **RF03** – Il sistema deve distinguere due soglie di staleness: dati tra 12 e 24 ore (`is_stale`, warning giallo) e dati oltre le 24 ore (`is_very_stale`, allarme rosso).
- **RF04** – Il sistema deve colorare dinamicamente i valori delle metriche in base a soglie predefinite (verde/arancio/rosso).
- **RF05** – Il sistema deve permettere la navigazione al dettaglio di ogni arnia tramite click sulla card della dashboard.
- **RF06** – Il sistema deve visualizzare grafici storici con finestre temporali selezionabili (24h, 7 giorni, 30 giorni, 1 anno).
- **RF07** – Il sistema deve calcolare e visualizzare il coefficiente di correlazione R^2 tra temperatura interna ed esterna.
- **RF08** – Il sistema deve recuperare e visualizzare gli allarmi attivi e storici da ThingsBoard, raggruppandoli per arnia.
- **RF09** – Il sistema deve tradurre i tipi di allarme ThingsBoard in etichette leggibili in italiano.
- **RF10** – Il sistema deve permettere all'utente di cambiare lo stato di un allarme (Da Gestire → Aperto → Risolto) tramite modale, con restrizione ai ruoli operator e admin.
- **RF11** – Il sistema deve sincronizzare lo stato degli allarmi con ThingsBoard
- **RF12** – Il sistema deve persistere gli stati locali degli allarmi nel browser tramite `localStorage`.
- **RF13** – Il sistema deve permettere l'aggiunta di note testuali a ogni allarme gestito.
- **RF14** – Il sistema deve fornire una modalità demo attivabile da toggle, che sostituisce le chiamate API con dati statici predefiniti.
- **RF15** – Il sistema deve visualizzare grafici di analisi avanzata: scatter plot temperatura In/Out, derivata prima della temperatura, derivata prima del peso aggregato.
- **RF16** – Il sistema deve calcolare statistiche aggregate dell'apiario: peso totale miele, temperatura media, umidità media, conteggio allarmi attivi.
- **RF17** – Il sistema deve implementare una cache di sessione PHP di 5 secondi per le risposte aggregate.
- **RF18** – Il sistema deve recuperare un fallback "latest" quando non sono disponibili dati nelle ultime 24 ore.
- **RF19** – Il sistema deve esporre una pagina archivio separata per la consultazione degli allarmi risolti.

- **RF20** – Il sistema deve permettere agli admin di creare, modificare il ruolo ed eliminare gli account utente.
- **RF21** – Il sistema deve permettere agli admin di reimpostare la password di qualsiasi altro utente.
- **RF22** – Il sistema deve permettere agli admin di “impersonare” qualsiasi altro utente, con banner visivo di avviso e funzione di ripristino.
- **RF23** – Il sistema deve permettere a ogni utente di modificare la propria password dalla pagina profilo.
- **RF24** – Il sistema deve aggiornare i dot di stato nella sidebar ogni 30 secondi tramite polling indipendente.

12. Requisiti Non Funzionali

12.1 Prestazioni

Il backend PHP implementa due livelli di cache per ridurre il carico su ThingsBoard: il token JWT viene mantenuto in sessione per 3600 secondi (1 ora), mentre le risposte aggregate vengono mantenute in cache per 5 secondi. Le richieste telemetriche per tutte le arnie vengono eseguite in parallelo tramite `curl_multi`, riducendo il tempo di risposta complessivo rispetto alle chiamate sequenziali. Il frontend esegue le chiamate API in modo asincrono (`async/await`) per non bloccare il rendering dell'interfaccia.

12.2 Sicurezza

La sicurezza è implementata su più livelli:

- Autenticazione a due livelli: HTTP Basic Auth (`.htaccess`) e login PHP con sessioni.
- Hashing BCrypt delle password tramite `password_hash` e `password_verify`.
- Le credenziali di ThingsBoard sono definite in `config.php` e non vengono mai esposte al client.
- Il controllo dei ruoli è applicato sia lato client (nascondendo elementi UI non accessibili) sia lato server tramite `requireRole()` e la verifica esplicita del ruolo in `alarm_action.php` (risposta 403 per i viewer).
- Le query al database MySQL utilizzano prepared statements con bind di parametri per prevenire SQL injection.

12.3 Scalabilità

L'interfaccia è responsive e utilizzabile da desktop, tablet e smartphone grazie a Bootstrap 5.3.2. I breakpoint CSS sono definiti per schermi inferiori a 992px e 576px, con adattamenti specifici (colonne impilate, dimensioni ridotte, altezze fisse ridotte). La modalità demo permette di effettuare presentazioni o test dell'interfaccia in qualsiasi contesto, senza dipendenza dall'hardware. I messaggi di feedback (badge colorati, semafori) sono progettati per comunicare lo stato del sistema in modo immediato e intuitivo, senza richiedere conoscenze tecniche specifiche da parte dell'utente finale.

13. Conclusione

Il progetto Smart BeeHive ha permesso di realizzare una piattaforma completa per il monitoraggio remoto di un apiario intelligente, integrando tecnologie hardware e software in un unico ecosistema. Attraverso la raccolta e l'analisi dei dati provenienti dalle arnie, il sistema fornisce agli apicoltori uno strumento concreto per controllare lo stato delle colonie e individuare tempestivamente eventuali anomalie.

Dal punto di vista tecnico, lo sviluppo del progetto ha richiesto l'integrazione di diverse tecnologie, tra cui dispositivi IoT per l'acquisizione dei dati, un backend per l'elaborazione delle informazioni, comunicazioni LoRa e l'utilizzo di ThingsBoard per l'archiviazione e la gestione dei dati.

Un aspetto particolarmente significativo è stato lo sviluppo di un sistema capace di trasformare i dati grezzi raccolti dai sensori in informazioni chiare e immediatamente interpretabili dall'utente, grazie a una dashboard interattiva con grafici storici e strumenti di analisi avanzata. Questo ha reso più efficace la consultazione delle informazioni rilevanti e ha migliorato la capacità di individuare variazioni e criticità nel tempo.

In conclusione, Smart Hive rappresenta un esempio concreto di applicazione delle competenze informatiche, evidenziando come le tecnologie digitali possano contribuire a rendere più efficiente e innovativa la gestione del monitoraggio nel settore dell'apicoltura.

Smart Hive rappresenta un esempio concreto di applicazione delle competenze informatiche, dimostrando come le tecnologie digitali possano contribuire a rendere più efficiente e innovativa la gestione delle attività di monitoraggio nel settore dell'apicoltura.
